



Detecting anomalies in network logs through their visualization





TABLE OF CONTENTS



01 Goals

02 Timeline

03 Concepts

04 Similarity Matrix

05 HiddenStateVis

06 T-SNE

07 DATASET

08 Conclusion



01 goals





GOALS

GOAL 1

**Network
Security
Monitoring**



Zero-day attacks

GOAL 2

**Anomaly
Detection**



future



GOAL 3

**Capacity
Planning**



GOAL 4

**Network
Planning and
Design**



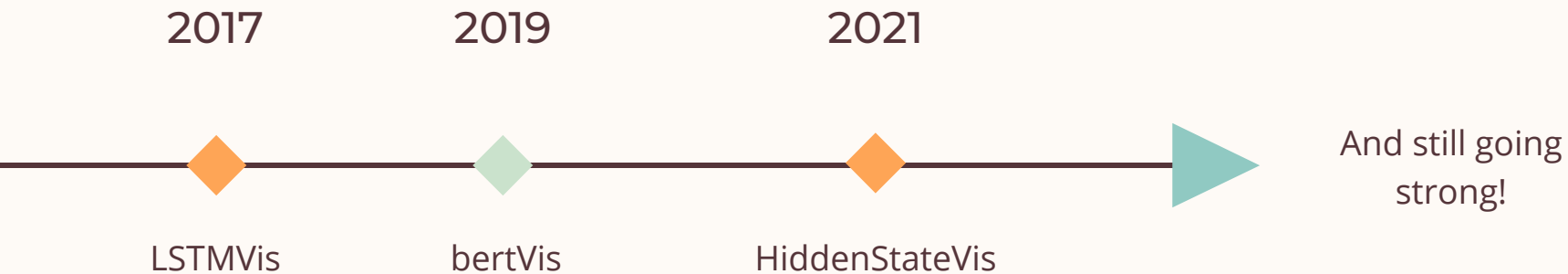


OUR EVOLUTION





OUR EVOLUTION



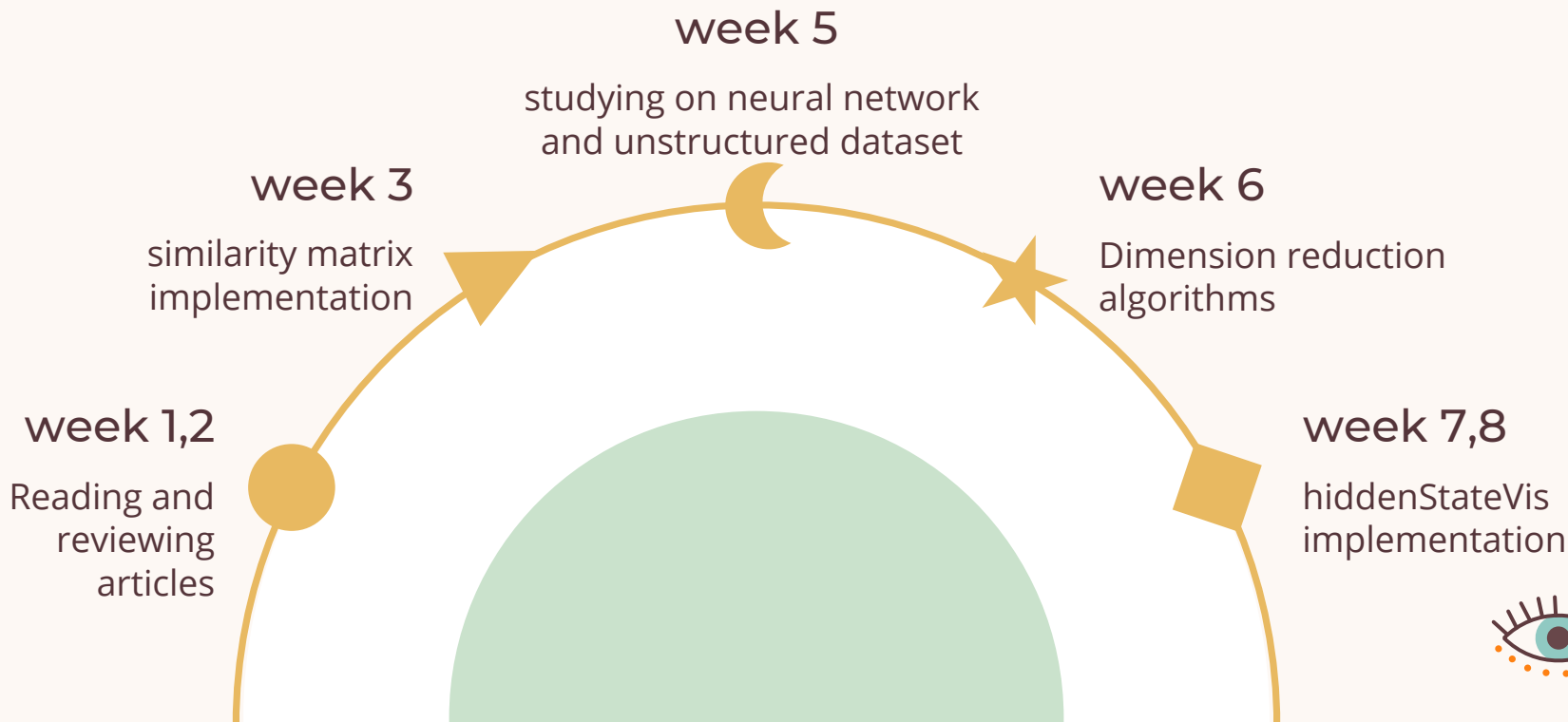


02 timeline





TIMELINE







Network log

"network logs" refer to a record or documentation of activities and events that occur within a **network**. These logs are typically generated and maintained by **network devices** and security systems to monitor and track various network activities.





04 Similarity matrix



Visualization with Machine Learning Model



DATASET

NSL-KDD

[train model](#)



Prediction

Svm model

[Prediction on test
set](#)



VISUALIZATION

Similarity matrix

[Visualization](#)



05 Hidden state visualization



Deep Learning Visualization



RNN

Accuracy :
80.5 %



Dense

Accuracy :
75.5 %



LSTM

Accuracy :
99.25 %



CNN

Accuracy :
98.5 %



GRU

Accuracy :
99.25 %





06 T-SNE





T-SNE



T-distributed Stochastic Neighbor Embedding

t-SNE is a tool to visualize high-dimensional data

t-SNE takes a **high-dimensional dataset** as input and reduces it to a lower-dimensional representation, typically **two or three dimensions**. Reducing the dimensionality makes it easier to visualize and analyze the data while retaining its essential characteristics.



Implementation

```
summary of model_lstm  
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 10, 32)	30432
lstm_1 (LSTM)	(None, 10, 100)	53200

```
=====  
Total params: 83632 (326.69 KB)  
Trainable params: 83632 (326.69 KB)  
Non-trainable params: 0 (0.00 Byte)
```

```
summary of model_pred  
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
dropout_1 (Dropout)	(None, 100)	0
dense_4 (Dense)	(None, 1)	101

```
=====  
Total params: 101 (404.00 Byte)  
Trainable params: 101 (404.00 Byte)
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 100, 32)	2834784
lstm (LSTM)	(None, 100)	53200
dropout (Dropout)	(None, 100)	0
dense (Dense)	(None, 1)	101

```
=====  
Total params: 2888085 (11.02 MB)  
Trainable params: 2888085 (11.02 MB)  
Non-trainable params: 0 (0.00 Byte)
```





DATASET





unstructured vs structured data

Id	Content	EventId	EventTemplate
774	Notification time out: 3200	E31	Notification time out: <*>
493	Received connection request /10.10.34.11:45307	E40	Received connection request /<*>:<*>
688	Send worker leaving thread	E42	Send worker leaving thread
679	Interrupted while waiting for message on queue	E24	Interrupted while waiting for message on queue
688	Send worker leaving thread	E42	Send worker leaving thread
762	Connection broken for id 188978561024, my id = 1, error =	E11	Connection broken for id <*>, my id = <*>, error =
493	Received connection request /10.10.34.13:57707	E40	Received connection request /<*>:<*>



Zookeeper dataset

```
: df.head(5)
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_srv
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	25	0.17	
1	0	udp	other	SF	146	0	0	0	0	0	...	1	0.00	
2	0	tcp	private	S0	0	0	0	0	0	0	...	26	0.10	
3	0	tcp	http	SF	232	8153	0	0	0	0	...	255	1.00	
4	0	tcp	http	SF	199	420	0	0	0	0	...	255	1.00	

5 rows x 42 columns



NSL-KDD



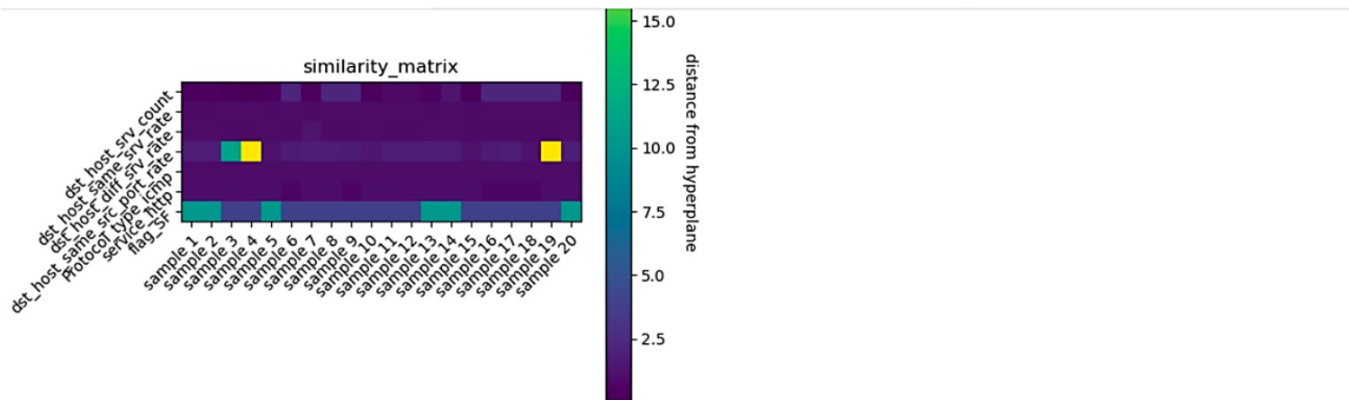


conclusion





Similarity matrix



<Figure size 5000x1000 with 0 Axes>

```
log_dataset.head(10)
```

	src_bytes	dst_bytes	hot	count	srv_count	same_srv_rate	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_srv_rate	dst_host_same_src_port_rate
0	-0.021988	-0.096896	-0.113521	1.166764	-0.237191	-1.697859	-1.169697	-1.305370	-0.138370	-0.431856
1	-0.021988	-0.096896	-0.113521	0.443234	-0.338246	-1.770589	-1.250212	-1.397181	-0.138370	-0.431856
2	0.005473	-0.096896	-0.113521	-0.607053	-0.338246	0.629488	-0.489800	0.002934	-0.228985	1.559906
3	-0.021946	-0.096896	-0.113521	-0.607053	0.380366	0.629488	-0.749234	0.898090	-0.410217	2.833328
4	-0.021988	-0.096189	-0.113521	-0.607053	-0.259648	0.629488	-0.489800	-0.685647	0.360018	-0.333901
5	-0.021423	0.587166	-0.113521	-0.583713	-0.304561	0.629488	1.022079	0.898090	-0.410217	-0.399204
6	-0.019826	-0.078657	-0.113521	-0.607053	-0.315789	0.629488	-1.008669	-1.144701	2.851953	-0.431856
7	-0.021715	-0.088696	-0.113521	-0.607053	-0.338246	0.629488	1.022079	0.898090	-0.410217	-0.431856
8	-0.021296	-0.074887	-0.113521	-0.358096	0.178256	0.629488	1.022079	0.898090	-0.410217	-0.399204
9	-0.021933	-0.089497	-0.113521	-0.607053	-0.338246	0.629488	-1.026561	-0.249546	-0.047754	-0.366553





Neural network Layers



Dense Layer

each neuron receives input from all the neurons of previous layer



RNN Layer

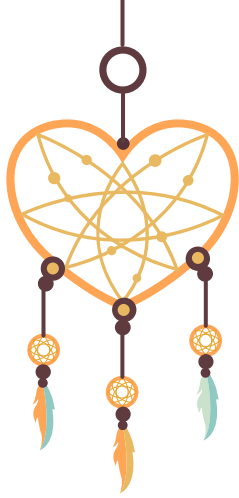
use the recurrent unit to store the previous state



LSTM Layer

Use multiple units to make it more efficient





THANKS!

Do you have any questions?

This presentation template was
created by **Slidesgo**, including icons
by **Flaticon**, and infographics &
images by **Freepik**